Budgify

Development Documentation

Version:	2.1 -- Implementation Aligned
Date:	April 2026
Framework:	ASP.NET Core 8 Web API
Database:	SQL Server (SQLEXPRESS) via EF Core 8
Auth:	JWT Bearer (HttpOnly app.at cookie)
Libraries:	AutoMapper · CsvHelper · Serilog

Personal Finance Management Application

Includes: Data Model · CSV Import Architecture · API Specification · Implementation Roadmap

1. Project Overview

1.1 Purpose

Budgify is a personal finance management application that enables users to track income and expenses across multiple wallets, plan budgets using percentage-based allocation, import bank statement CSVs, and gain insights into their spending behavior. This document serves as the single source of truth for the application's data architecture, feature specifications, and implementation plan.

1.2 Core Capabilities

- Multi-wallet management (checking, savings, credit card, cash accounts)
- Manual transaction entry and CSV bank statement imports
- Percentage-based budget planning with spending buckets
- Smart transaction categorization with learning capabilities [Planned]
- Fixed vs variable expense tracking
- Overspend detection and budget monitoring [Planned]
- Import batch tracking with batch approve/reject and rollback support

1.3 Technical Stack

> The original document was stack-agnostic (MySQL recommended). The implemented stack is ASP.NET Core 8 + SQL Server.

Layer	Technology
Framework	ASP.NET Core 8 Web API
ORM	Entity Framework Core 8 (Code-First, Migrations)
Database	SQL Server (SQLEXPRESS)
Auth	JWT Bearer -- token as app.at HttpOnly cookie, sent as Authorization: Bearer
Object Mapping	AutoMapper
Logging	Serilog
CSV Parsing	CsvHelper 33.1.0
Password Hashing	ASP.NET Identity PasswordHasher<User>

All IDs are GUIDs (v4). All timestamps are UTC.

> URL versioning removed: The original spec used /api/v1/ throughout. The implemented API uses unversioned routes: /api/

2. Data Model

All entities include soft-delete support via a DeletedAt nullable timestamp, except SpendingBucket and BudgetAllocation which are managed without soft-delete in the current implementation.

2.1 User [Done]

The original spec had a single name field. Implementation splits this into FirstName and LastName. Settings is stored as a JSON string (not JSONB). Email is used for login -- there is no separate username field.

Field	Type	Required	Description
Id	Guid	PK	Auto-generated unique identifier.
FirstName	string(50)	Yes	First name. (was: single 'name' field)
LastName	string(50)	Yes	Last name. (was: single 'name' field)
Email	string(255)	Yes	Login email. Must be unique.
PasswordHash	string	Yes	ASP.NET Identity PasswordHasher. Never plaintext.
Currency	string	Yes	Preferred currency code. Default: USD.
Settings	string?	No	JSON string for user preferences.
CreatedAt	DateTime	Auto	UTC.
UpdatedAt	DateTime?	Auto	UTC.
DeletedAt	DateTime?	No	Soft-delete timestamp. Null if active.

2.2 Wallet [Done]

Field	Type	Required	Description
Id	Guid	PK	Auto-generated.
UserId	Guid (FK)	Yes	References User. Owner of this wallet.
WalletName	string(100)	Yes	User-defined name (e.g., 'Chase Checking').
WalletType	WalletType (enum)	Yes	Stored as string. See Appendix.
Institution	string(100)?	No	Bank name. Aids CSV format detection.
Balance	decimal(12,2)	Yes	Current balance. Updated via delta on every transaction.
Currency	Currency (enum)	Yes	Stored as string. Inherits from User by default.
LastSynced	DateTime?	No	Last CSV import timestamp.
CreatedAt	DateTime	Auto	UTC.

UpdatedAt	DateTime	Auto	UTC.
DeletedAt	DateTime?	No	Soft-delete timestamp.

Note: Balance is maintained via delta adjustments -- not recalculated from all transactions. Rollback reverses only the delta of confirmed transactions.

2.3 Transaction [Done]

New in Phase 2.8: IsDuplicate (bool) and ImportStatus (nullable enum) added to support the import review workflow.

Field	Type	Required	Description
Id	Guid	PK	Auto-generated.
WalletId	Guid (FK)	Yes	References Wallet.
UserId	Guid (FK)	Yes	Denormalized for fast permission checks.
SubcategoryId	Guid? (FK)	No	References TransactionCategory. Null = uncategorized.
ImportId	Guid?	No	References ImportBatch. Null for manual transactions.
Amount	decimal(12,2)	Yes	Always positive. Direction set by TransactionType.
TransactionType	Enum	Yes	Income or Expense. Stored as string.
Date	DateOnly	Yes	Posting date for CSV imports.
Description	string(500)?	No	User-facing (cleaned) description.
RawDescription	string(1000)?	No	Original bank description, preserved as-is.
BankTxnType	string(50)?	No	Bank type code (e.g., DEBIT_CARD, ACH_CREDIT).
BalanceAfter	decimal(12,2)?	No	Running balance from bank statement. Reconciliation only.
CheckNumber	string(20)?	No	Check or slip number if applicable.
IsRecurring	bool	Yes	Default: false.
RecurringRuleId	Guid?	No	Reserved for future RecurringRule feature.
DuplicateHash	string(64)?	No	SHA-256 hex string. See Section 4.5.
Source	TransactionSource	Yes	Manual or Import. Stored as string. Default: Manual.
IsDuplicate [NEW]	bool	Yes	Set true during import when hash matches existing transaction.
ImportStatus [NEW]	ImportStatus?	No	Null for manual. Pending/Confirmed/Rejected for imports.
CreatedAt	DateTime	Auto	UTC.
UpdatedAt	DateTime	Auto	UTC.

DeletedAt	DateTime?	No	Soft-delete timestamp.

Indexes: (UserId, Date) . (WalletId, Date) . DuplicateHash

2.4 TransactionCategory [Done]

New fields added: Icon (string) and Color (hex string) for UI rendering -- not in original spec.

Field	Type	Required	Description
Id	Guid	PK	Auto-generated.
BucketId	Guid (FK)	Yes	References SpendingBucket.
UserId	Guid? (FK)	No	Null = system category. Set = user-created.
CategoryName	string(100)	Yes	Display name (e.g., 'Groceries', 'Netflix').
IsFixed	bool	Yes	True for fixed expenses. Default: false.
Icon [NEW]	string(50)?	No	Icon identifier for UI rendering.
Color [NEW]	string(7)?	No	Hex color code, e.g., #FF5733.
CreatedAt	DateTime	Auto	UTC.
DeletedAt	DateTime?	No	Soft-delete. System categories cannot be deleted.

2.5 SpendingBucket [Done -- Simplified]

Simplified: Original spec included user_id, is_custom, icon, sort_order, deleted_at. The current implementation only stores Id and BucketName. All buckets are system-managed (seeded). User-created custom buckets are planned for Phase 5.

Field	Type	Required	Description
Id	Guid	PK	Unique identifier.
BucketName	string(100)	Yes	Display name (e.g., 'Housing', 'Lifestyle').
user_id [REMOVED]	Guid?	--	Removed. All buckets are system-managed.
is_custom [REMOVED]	bool	--	Removed.
icon [REMOVED]	string	--	Removed.
sort_order [REMOVED]	int	--	Removed.
deleted_at [REMOVED]	DateTime?	--	Removed. Buckets are not soft-deleted.

2.6 Budget [Done]

Field	Type	Required	Description
Id	Guid	PK	Auto-generated.
UserId	Guid (FK)	Yes	References User. Owner of this budget.
Name	string(100)	Yes	User-defined label (e.g., 'March 2026').
TotalIncome	decimal(12,2)	Yes	Total income being budgeted.
StartDate	DateOnly	Yes	First day of budget period.
EndDate	DateOnly	Yes	Last day of budget period.
IsActive	bool	Yes	Only one active budget per user. Default: false.
CreatedAt	DateTime	Auto	UTC.
UpdatedAt	DateTime	Auto	UTC.
DeletedAt	DateTime?	No	Soft-delete timestamp.

Constraint: Partial unique index on (UserId) where IsActive = 1. Enforced at the database level by EF Core migration.

2.7 BudgetAllocation [Done]

> Constraint relaxed: Original spec required allocations to sum to exactly 100%. The implementation validates that total does not exceed 100% -- it is not required to equal exactly 100%.

Field	Type	Required	Description
Id	Guid	PK	Auto-generated.
BudgetId	Guid (FK)	Yes	References Budget (cascade delete).
BucketId	Guid (FK)	Yes	References SpendingBucket (cascade delete).
Percentage	decimal(5,2)	Yes	Percentage of total income allocated (e.g., 30.00).
AllocatedAmount	decimal(12,2)	Yes	Computed: (Percentage / 100) x Budget.TotalIncome.

2.8 ImportBatch [Done]

> Changes from original spec: Removed file_size_bytes, bank_format, date_range_start, date_range_end, error_message. Renamed completed_at to ConfirmedAt. Added RolledBackAt. Status simplified to 3 states.

Field	Type	Required	Description
Id	Guid	PK	Default: Guid.NewGuid().

UserId	Guid (FK)	Yes	References User. DeleteBehavior.Restrict (avoids EF1785).
WalletId	Guid (FK)	Yes	References Wallet.
FileName	string(255)	Yes	Original uploaded file name.
Status	ImportBatchStatus	Yes	Reviewing, Confirmed, or RolledBack.
TotalRows	int	Yes	Total rows parsed from the CSV.
ImportedCount	int	Yes	Count of confirmed transactions.
DuplicateCount	int	Yes	Rows flagged as duplicates on upload.
SkippedCount	int	Yes	Rejected transactions after confirm.
file_size_bytes [REMOVED]	int	--	Removed from implementation.
bank_format [REMOVED]	enum	--	Removed. Generic CSV parser used.
date_range_start [REMOVED]	Date	--	Removed.
date_range_end [REMOVED]	Date	--	Removed.
error_message [REMOVED]	string	--	Removed. Errors returned as HTTP responses.
CreatedAt	DateTime	Auto	UTC.
ConfirmedAt (was: completed_at)	DateTime?	No	Set when batch is confirmed.
RolledBackAt [NEW]	DateTime?	No	Set when batch is rolled back.

2.9 CategorizationRule [Planned -- Phase 4]

> This entity is defined in the spec but has not yet been implemented. Planned for Phase 4.

Will define pattern-matching rules for automatically assigning categories to imported transactions. Fields remain as originally specified (rule_id, user_id, match_pattern, match_type, subcategory_id, priority, source, match_count, is_active, created_at).

2.10 RecurringRule [Future -- Phase 5]

> Not yet implemented. The IsRecurring flag and RecurringRuleId on Transaction are placeholders. Fields remain as originally specified.

3. Entity Relationship Summary

From Entity	To Entity	FK Field	Cardinality
Wallet	User	UserId	Many-to-One
Transaction	Wallet	WalletId	Many-to-One
Transaction	User	UserId	Many-to-One
Transaction	TransactionCategory	SubcategoryId	Many-to-One (nullable)
Transaction	ImportBatch	ImportId	Many-to-One (nullable)
TransactionCategory	SpendingBucket	BucketId	Many-to-One
TransactionCategory	User	UserId	Many-to-One (nullable)
Budget	User	UserId	Many-to-One
BudgetAllocation	Budget	BudgetId	Many-to-One (cascade delete)
BudgetAllocation	SpendingBucket	BucketId	Many-to-One (cascade delete)
ImportBatch	User	UserId	Many-to-One (RESTRICT cascade)
ImportBatch	Wallet	WalletId	Many-to-One
CategorizationRule [Planned]	User	user_id	Many-to-One (nullable)
CategorizationRule [Planned]	TransactionCategory	subcategory_id	Many-to-One

ImportBatch -> User uses DeleteBehavior.Restrict to avoid SQL Server error 1785 (multiple cascade paths to the Users table).

4. CSV Import Specification

4.1 Supported Formats

Current implementation uses a generic named-column approach. Bank-specific parsers are planned for Phase 5.

Bank / Format	Status	Notes
Generic (named CSV headers)	Done	CSV must have named columns matching the mapping below.
Chase (auto-detect)	Planned	Phase 5.
Bank of America	Planned	Phase 5.
Wells Fargo	Planned	Phase 5.

4.2 CSV Column Mapping (Generic Format)

The original spec defined Chase-specific column mapping. Current implementation uses generic named columns via CsvHelper with MissingFieldFound = null (optional columns silently skipped).

CSV Column (header)	Transaction Field	Required	Notes
Date	Date	Yes	Parsed as DateOnly.
Amount	Amount	Yes	Stored as absolute positive value.
Type	TransactionType	Yes	Income or Expense.
Description	Description	No	User-facing description.
RawDescription	RawDescription	No	Original bank description.
BankTxnType	BankTxnType	No	Bank-provided type label.
CheckNumber	CheckNumber	No	
BalanceAfter	BalanceAfter	No	Decimal. Running balance from bank.

4.3 Chase Transaction Type Codes

For reference when Chase-specific parsing is implemented (Phase 5):

Type Code	Direction	Description
DEBIT_CARD	Expense	Point-of-sale card purchase.
ACH_CREDIT	Income	Direct deposit / ACH inflow.
ACH_DEBIT	Expense	Automated payment / ACH outflow.
LOAN_PMT	Expense	Credit card or loan payment.

ACCT_XFER	Either	Internal bank transfer. Exclude from budget.
QUICKPAY_DEBIT	Expense	Zelle outbound.
QUICKPAY_CREDIT	Income	Zelle inbound.
MISC_DEBIT	Expense	Other outflow (investments, transfers).
MISC_CREDIT	Income	Other inflow.

4.4 Description Cleaning Pipeline

Applies to Chase-specific parsing (Phase 5). The generic parser stores Description column values directly.

1. Strip trailing dates (e.g., ' 03/23' at end of DEBIT_CARD descriptions).
2. Extract merchant name for DEBIT_CARD (text before state abbreviation).
3. Parse ACH descriptions: extract ORIG CO NAME value.
4. Parse transfer descriptions: extract destination from ACCT_XFER entries.
5. Parse Zelle descriptions: extract recipient/sender name.
6. Parse LOAN_PMT: extract card ending digits.
7. Title-case the result.
8. Always preserve RawDescription for reference and re-parsing.

4.5 Duplicate Detection Algorithm

> Hash formula updated: The original spec hashed date + amount + raw_description. The implementation includes walletId as the first component to scope duplicates per wallet.

```
input = "{walletId}|{date:yyyy-MM-dd}|{amount}|{rawDescription?.ToLower().Trim() ?? ""}"
hash  = SHA256(UTF8.GetBytes(input)).ToHexString().ToLower()   // 64-char hex
```

Duplicate rows are NOT auto-skipped. They are flagged with IsDuplicate = true and surfaced to the reviewer. The reviewer uses batch approve/reject to decide what to include.

4.6 Import Workflow State Machine

> States simplified: Original spec had 5 states (PENDING, REVIEWING, COMPLETED, ROLLED_BACK, FAILED). Implementation uses 3 states. Parse errors are returned as HTTP 400 before a batch is created.

From	To	Trigger	Action
(new)	Reviewing	POST /api/import/wallets/{id}	Batch + pending transactions created. Duplicates flagged.
Reviewing	Reviewing	PATCH approve / reject	ImportStatus updated on selected transactions.
Reviewing	Confirmed	POST confirm	Rejected txns soft-deleted. Remaining confirmed. Balance adjusted.
Confirmed	RolledBack	POST rollback	Balance delta reversed. All confirmed txns soft-deleted.

A Reviewing batch cannot be rolled back. It must be confirmed first, then rolled back.

5. Auto-Categorization Engine [Planned -- Phase 4]

> The auto-categorization engine is not yet implemented. This section describes the planned design.

5.1 How It Works

When transactions are imported via CSV, the engine runs each transaction's RawDescription through the CategorizationRule table to attempt automatic category assignment, reducing manual work after each import.

5.2 Rule Evaluation

1. Load all active rules for the user, plus all system rules (user_id IS NULL).
2. Sort by priority DESC, then source rank (USER_CREATED=3 > LEARNED=2 > SYSTEM=1).
3. For each imported transaction, iterate rules in sorted order.
4. Apply match based on match_type: CONTAINS, STARTS_WITH, EXACT, REGEX.
5. First matching rule wins. Set SubcategoryId; increment match_count.
6. If no rule matches, leave SubcategoryId null and flag for manual review.

5.3 Learning from User Actions

When a user manually categorizes an uncategorized transaction, the system will prompt: "Always categorize transactions containing 'LA FITNESS' as Health & Fitness?" If accepted, a new rule with source = LEARNED is created using the cleaned description keyword.

5.4 System Default Rules

Pattern	Match Type	Category	Bucket
NETFLIX	CONTAINS	Streaming Services	Lifestyle
LA FITNESS	CONTAINS	Gym & Fitness	Lifestyle
LINKEDIN	CONTAINS	Professional Subscriptions	Lifestyle
REMITLY	CONTAINS	Remittance	Other
PAYROLL	CONTAINS	Salary	Income
WEALTHFRONT	CONTAINS	Investments	Savings & Investments
SOFI	CONTAINS	Savings Transfer	Savings & Investments
Payment to Chase card	STARTS_WITH	Credit Card Payment	Debt
STUDENT LOAN	CONTAINS	Student Loan	Debt
Zelle payment	STARTS_WITH	Peer Payment	Other

6. API Endpoint Specification

URL change: All routes use /api/ -- not /api/v1/ as originally specified. Authentication via Authorization: Bearer <token> extracted from the app.at cookie.

6.1 Authentication -- /api/authentication

Route changed: Original was /api/v1/auth/. Implemented as /api/authentication/. Login endpoint is /authenticate (not /login). Refresh and password-reset endpoints are not yet implemented.

Method	Endpoint	Auth	Description
POST	/api/authentication/register	No	Register. Body: { firstName, lastName, email, password, currency }.
POST	/api/authentication/authenticate	No	Login. Sets app.at cookie. Body: { email, password }.
POST [PLANNED]	/api/authentication/refresh	Yes	Refresh expiring JWT token.
POST [PLANNED]	/api/authentication/forgot-password	No	Initiate password reset via email.
POST [PLANNED]	/api/authentication/reset-password	No	Complete password reset with token.

6.2 Wallets -- /api/wallets

Method	Endpoint	Description
GET	/api/wallets	Get authenticated user + their wallets array.
POST	/api/wallets	Create wallet. Body: { walletName, walletType, institution, currency, balance }.
GET	/api/wallets/{id}	Get wallet details.
PUT	/api/wallets/{id}	Update wallet fields.
DELETE	/api/wallets/{id}	Soft-delete wallet.

6.3 Transactions -- /api/transactions

Method	Endpoint	Description
GET	/api/transactions?walletId={guid}	List transactions for a wallet. Excludes soft-deleted.

POST	/api/transactions	Create manual transaction. Adjusts wallet balance. Body: { walletId, amount, transactionType, date, description, subcategoryId }.
GET	/api/transactions/{id}	Get single transaction detail.
PUT	/api/transactions/{id}	Partial update. Null fields skipped via AutoMapper PreCondition.
DELETE	/api/transactions/{id}	Soft-delete. Reverses wallet balance adjustment.

6.4 Transaction Categories -- /api/transactioncategories

> Route is /api/transactioncategories (not /api/categories as in the original spec).

Method	Endpoint	Description
GET	/api/transactioncategories	List all categories: system + user-owned. Excludes soft-deleted.
GET	/api/transactioncategories/{id}	Get single category.
POST	/api/transactioncategories	Create user category. Body: { bucketId, categoryName, isFixed, icon, color }.
PUT	/api/transactioncategories/{id}	Update category fields.
DELETE	/api/transactioncategories/{id}	Soft-delete. System categories are blocked from deletion.

6.5 CSV Import -- /api/import

Route and flow changed from original spec. Old: /api/v1/imports/ with preview endpoints. New: /api/import/ with streamlined upload -> review -> approve/reject -> confirm/rollback flow.

Method	Endpoint	Description
POST	/api/import/wallets/{walletId}	Upload CSV (multipart/form-data, field: file). Returns 201 ImportBatchDto.
GET	/api/import	List all import batches for authenticated user.
GET	/api/import/{id}	Get batch detail: transactions, pendingDuplicates, readyToConfirm.
PATCH	/api/import/{id}/transactions/approve	Bulk-approve transactions. Body: { transactionIds: [guid,...] }.
PATCH	/api/import/{id}/transactions/reject	Bulk-reject transactions. Body: { transactionIds: [guid,...] }.
POST	/api/import/{id}/confirm	Finalize: soft-delete rejected, confirm remaining, adjust wallet balance.
POST	/api/import/{id}/rollback	Reverse confirmed batch: reverse balance, soft-delete all confirmed txns.
GET [REMOVED]	/api/v1/imports/:id/preview	Removed. Combined into GET /api/import/{id}.
PATCH [REMOVED]	/api/v1/imports/:id/preview	Removed. Replaced by bulk approve/reject endpoints.

6.6 Budgets -- /api/budgets

Method	Endpoint	Description
GET	/api/budgets	List all budgets for authenticated user.
POST	/api/budgets	Create budget. Body: { name, totalIncome, startDate, endDate, isActive }.
GET	/api/budgets/{id}	Get budget with its allocations.
PUT	/api/budgets/{id}	Update budget fields.
DELETE	/api/budgets/{id}	Soft-delete budget.
GET	/api/budgets/{id}/allocations	List allocations for a budget.
POST	/api/budgets/{id}/allocations	Create allocation. Body: { bucketId, percentage, allocatedAmount }. Validates total <= 100%.

GET	/api/budgets/{id}/allocations/{aid}	Get single allocation.
PUT	/api/budgets/{id}/allocations/{aid}	Update allocation.
DELETE	/api/budgets/{id}/allocations/{aid}	Delete allocation.
GET [PLANNED]	/api/budgets/{id}/progress	Budget vs actual comparison with overspend alerts.

6.7 Spending Buckets

No CRUD endpoints yet. SpendingBuckets are seeded at startup. User-created custom buckets and bucket CRUD are planned for Phase 5.

6.8 Categorization Rules [Planned -- Phase 4]

Not yet implemented. Routes will be under /api/rules.

Method	Endpoint	Description
GET	/api/rules	List all rules (user + system).
POST	/api/rules	Create a rule.
PATCH	/api/rules/{id}	Update rule.
DELETE	/api/rules/{id}	Delete rule.
POST	/api/rules/test	Test a pattern against a description.

7. Use Case Catalog

7.1 User Account Management

- UC-001: Register new account with email, first name, last name, password, and preferred currency.
- UC-002: Sign in with email and password. JWT delivered via app.at cookie.
- UC-003: Reset forgotten password via email link. [Planned]
- UC-004: Update profile (name, currency, settings). [Planned]
- UC-005: Delete account. [Planned]

7.2 Wallet Management

- UC-010: Create a new wallet specifying name, type, institution, currency, and initial balance.
- UC-011: View all wallets with current balances.
- UC-012: Update wallet details (name, institution).
- UC-013: Soft-delete a wallet.

7.3 Transaction Management

- UC-020: Add a manual transaction with amount, date, type, description, wallet, and category.
- UC-021: View transactions filtered by wallet.
- UC-022: View transaction detail including raw and clean descriptions.
- UC-023: Update a transaction's category, description, amount, or date.
- UC-024: Soft-delete a transaction. Wallet balance recalculates.
- UC-025: Bulk-categorize multiple transactions. [Planned]

7.4 CSV Import

- UC-030: Upload a CSV bank statement file targeting a specific wallet.
- UC-031: System parses CSV, computes duplicate hashes, and flags matches.
- UC-032: Review batch: see all transactions, duplicate count, pendingDuplicates.
- UC-033: Bulk-approve duplicate-flagged transactions to include them in the import.
- UC-034: Bulk-reject duplicate-flagged transactions to exclude them.
- UC-035: Confirm batch: wallet balance adjusts, rejected transactions soft-deleted.
- UC-036: View import history showing all past batches with stats.
- UC-037: Rollback a confirmed import: balance reversed, all confirmed txns soft-deleted.
- UC-038: Auto-categorize transactions using CategorizationRule engine. [Planned]
- UC-039: Prompt to create a learned rule when user manually categorizes. [Planned]

7.5 Budget Planning

- UC-040: Create a named budget with total income, date range, and percentage allocations.
- UC-041: View budget allocations per spending bucket.
- UC-042: View budget vs actual spending per bucket. [Planned]
- UC-043: Receive overspend alert when a bucket exceeds allocation. [Planned]
- UC-044: Edit budget: adjust percentages, income, or date range.
- UC-045: Duplicate a previous budget as a template. [Planned]

7.6 Categorization Management

- UC-050: Create a custom transaction category under a spending bucket.
- UC-051: Tag a category as fixed or variable expense.
- UC-052: View, create, edit, and delete auto-categorization rules. [Planned]
- UC-053: Test a rule pattern against sample descriptions. [Planned]

8. Database Indexes & Constraints

8.1 Implemented Indexes

Table	Index Columns	Purpose
Transactions	(UserId, Date)	Primary listing query by date.
Transactions	(WalletId, Date)	Wallet-scoped transaction listing.
Transactions	DuplicateHash	Import deduplication lookup.
Budgets	(UserId) WHERE IsActive = 1	Partial unique index -- one active budget per user.

8.2 Key Constraints

- Budgets: Partial unique index on (UserId) filtered by IsActive = 1. Database-level via EF Core migration.
- BudgetAllocations: Application-level check that SUM(Percentage) <= 100% per budget. (Not required to equal exactly 100%.)
- Transactions.Amount: Always stored as a positive decimal. Direction determined by TransactionType.
- ImportBatch -> User (UserId): DeleteBehavior.Restrict to prevent SQL Server cascade cycle error 1785.

9. Seed Data

9.1 Default Spending Buckets

Bucket Name	Description
Housing	Rent, mortgage, property tax, home insurance, HOA fees.
Transportation	Car payment, gas, insurance, public transit, ride-share.
Lifestyle	Groceries, dining, entertainment, personal care, subscriptions.
Debt	Credit card payments, student loans, personal loans.
Savings & Investments	Savings transfers, 401(k), IRA, brokerage deposits.
Income	Salary, freelance, side hustles, interest income.
Other	Uncategorized, miscellaneous, remittances, peer payments.

9.2 Default Transaction Categories

Bucket	Category Name	Is Fixed	Example Transactions
Housing	Rent / Mortgage	Yes	Monthly rent, mortgage payment.
Housing	Utilities	Yes	Electric, water, gas, internet.
Housing	Home Insurance	Yes	Annual or monthly premium.
Transportation	Car Payment	Yes	Auto loan installment.
Transportation	Gas	No	Fuel purchases.
Transportation	Auto Insurance	Yes	Monthly premium.
Lifestyle	Groceries	No	Supermarket, grocery delivery.
Lifestyle	Dining Out	No	Restaurants, coffee shops, fast food.
Lifestyle	Entertainment	No	Movies, concerts, gaming.
Lifestyle	Streaming Services	Yes	Netflix, Spotify, Disney+.
Lifestyle	Gym & Fitness	Yes	Gym membership, fitness apps.
Lifestyle	Personal Care	No	Haircuts, salon, grooming.
Debt	Credit Card Payment	No	Monthly CC payments.
Debt	Student Loan	Yes	Monthly loan payment.
Savings & Investments	Savings Transfer	No	Transfers to savings accounts.
Savings & Investments	Investment Deposit	No	Brokerage, Wealthfront, M1.
Income	Salary	Yes	Regular paycheck.

Income	Freelance	No	Contract or gig income.
Other	Remittance	No	International money transfers (Remitly).
Other	Peer Payment	No	Zelle, Venmo, Cash App.

10. Implementation Roadmap

Phase 1: Foundation [Complete]

#	Task	Status
1.1	Project setup: EF Core, SQL Server, migrations	Done
1.2	Seed data: default buckets + categories	Done
1.3	User registration & authentication (JWT cookie)	Done
1.4	User profile CRUD	Done
1.5	Wallet CRUD	Done
1.6	Transaction CRUD (manual entry, balance adjustment)	Done
1.7	SpendingBucket (seeded, no CRUD endpoints yet)	Done
1.8	TransactionCategory CRUD (system + user-scoped, soft delete)	Done

Phase 2: Budget Planning [Complete]

#	Task	Status
2.1	Budget CRUD (one-active-per-user constraint)	Done
2.2	BudgetAllocation CRUD (nested under budget, <= 100% validation)	Done
2.3	Budget vs Actual aggregation query	Planned
2.4	Overspend detection per-bucket	Planned
2.5	Budget duplication (copy as template)	Planned

Phase 3: CSV Import [Complete]

#	Task	Status
3.1	Generic CSV parser (CsvHelper, MissingFieldFound=null)	Done
3.2	ImportBatch entity, migration, status tracking	Done
3.3	Duplicate detection via SHA-256 hash (includes walletId)	Done
3.4	IsDuplicate + ImportStatus fields added to Transaction	Done
3.5	Batch approve / reject endpoints (PATCH)	Done
3.6	Confirm endpoint (balance adjustment, soft-delete rejects)	Done
3.7	Rollback endpoint (delta reversal, soft-delete confirmed)	Done
3.8	Import history listing	Done
3.9	Chase-specific parser with header detection + description cleaning	Planned

Phase 4: Smart Categorization [Planned]

#	Task	Status
4.1	CategorizationRule entity and migration	Planned
4.2	Rule evaluation engine (match imported transactions)	Planned
4.3	Seed system default rules (Netflix, payroll, etc.)	Planned
4.4	Integrate auto-categorization into import upload	Planned
4.5	Learn from user corrections (prompt + create LEARNED rule)	Planned
4.6	Rule CRUD + test endpoint	Planned

Phase 5: Polish & Extend [Future]

#	Task	Status
5.1	Chase CSV parser (auto-detect headers, description cleaning)	Future
5.2	Bank of America CSV parser	Future
5.3	Wells Fargo CSV parser	Future
5.4	RecurringRule entity + transaction auto-generation	Future
5.5	Budget vs Actual monthly report	Future
5.6	Category spending trends over time	Future
5.7	SpendingBucket CRUD (user-created custom buckets)	Future
5.8	Data export (CSV and PDF)	Future
5.9	Transfer linking (pair send/receive transactions)	Future

11. Appendix

11.1 Enum Definitions

Enum	Values	Notes
WalletType	Checking, Savings, CreditCard, Cash, Loan, Investment	Stored as string in DB
Currency	USD, EUR, GBP, CAD, AUD, JPY, CHF, CNY, INR, MXN	Stored as string in DB
TransactionType	Income, Expense	Stored as string in DB
TransactionSource	Manual, Import	Stored as string in DB
ImportBatchStatus	Reviewing, Confirmed, RolledBack	Simplified from original 5-state model
ImportStatus	Pending, Confirmed, Rejected	Per-transaction import status -- new in Phase 2.8
MatchType	CONTAINS, STARTS_WITH, EXACT, REGEX	Planned -- CategorizationRule
RuleSource	SYSTEM, USER_CREATED, LEARNED	Planned -- CategorizationRule
Frequency	DAILY, WEEKLY, BIWEEKLY, MONTHLY, QUARTERLY, YEARLY	Future -- RecurringRule

11.2 Wallet Type Reference

Wallet Type	Examples	Notes
Checking	Chase Checking, everyday debit	Primary transaction wallet. Supports all types.
Savings	High-yield savings, emergency fund	Typically receives transfers.
CreditCard	Chase Sapphire, Amex Gold	Expenses only. Payments show as LOAN_PMT in checking.
Cash	Cash on hand, petty cash	Manual tracking only. No CSV import.
Loan	Student loan, mortgage	Tracks debt balance.
Investment	401(k), IRA, Wealthfront, M1	Tracks investment contributions and value.

11.3 Design Decisions & Gotchas

Decision	Detail
AutoMapper PreCondition	Use PreCondition (not Condition) for nullable value type fields in Update DTOs to skip mapping when source is null.

Never call .Update() on tracked entities	After FindAsync() / FirstOrDefaultAsync() the entity is already tracked. Modify properties and call SaveChangesAsync().
Bulk updates via ExecuteUpdateAsync	Used for batch approve/reject to update many rows without loading entities into memory.
Scoped repositories share DbContext	Repositories are Scoped; Services are Transient. Changes by one repo are visible to another within the same request.
Delta-based balance updates	Wallet balance is adjusted by a delta (+/- amount) -- never recalculated from all transactions. Rollback reverses only the delta of confirmed transactions.
FK cascade restriction	ImportBatch -> User (UserId) uses DeleteBehavior.Restrict to avoid SQL Server error 1785 (multiple cascade paths).
Soft deletes	Entities use DeletedAt timestamp. All queries filter DeletedAt == null.

11.4 Sample Chase CSV Observations

Key patterns from Chase9275_Activity_20260330.CSV (66 transactions, Jan-Mar 2026):

- Payroll deposits are bi-weekly ACH_CREDIT entries with 'TEXAS PRE PAYROLL' in description. Amount: $2,310.87.
- Investment transfers to Wealthfront and M1 Finance appear as MISC_DEBIT -- default to Savings & Investments.
- SoFi Bank appears as both MISC_DEBIT (outgoing) and MISC_CREDIT (incoming), indicating cross-bank movement.
- Remitly transactions use both DEBIT_CARD and MISC_DEBIT types. Match on 'REMITLY' or 'RMTLY'.
- Credit card payments (LOAN_PMT) reference card ending digits like '1850' and '6479'.
- Internal transfers (ACCT_XFER) reference 'CHK ...4060'. Exclude from budget calculations.
- Balance column may be empty on recent rows -- parser must handle missing balance gracefully.